Lecture 9：栈在括号匹配中的应用

9.1 括号匹配问题

9.1.1 基本概念

图示

- 核心：
  - 1. 输入左括号时，将其压栈；
  - 2. 输入右括号时，就对一个栈顶的左括号出栈；
  - 3. 处理完所有的输入括号后，当前栈非空 -> 也表示有括号没有匹配；
- 图示：



9.1.2 算法实现

栈结构

- 补充：在考试中，可以直接使用以下定义好的接口的名字，而且在卷子当中写的时候也可以写上其对应的中文含义注释；

```
//栈结构
#define MaxSize 10
typedef struct{
    char data[MaxSize];    //静态数组存放栈中元素
    int top;               //栈顶指针
}SqStack;

//初始化栈
void InitStack(SqStack &S)

//判断栈是否为空
bool StackEmpty(SqStack S)

//新元素入栈
bool Push(SqStack &S,char x)

//栈顶元素出栈，用x返回
bool Pop(SqStack &S,char &x)
```

算法实现

```
//函数算法
bool bracketCheck(char str[],int length)
{
    SqStack S;
    InitStack(S);    //初始化一个栈

    for(int i=0,i<length;i++)
    {
        if(str[i]=='(' || str[i]=='[' || str[i]=='{')
        {
            Push(S,str[i]);  // 扫描左括号，入栈
        }
        else // 扫描到右括号
        {
            if(StackEmpty(S)) // 当前栈空
                return false; // 匹配失败

            // 检查栈顶元素，是否和当前右括号匹配
            char topElem;
            topElem = Pop(S);   //栈顶元素出栈
            if(str[i]==')' && topElem!='(')
                return false;
            if(str[i]==']' && topElem!='[]')
                return false;
            if(str[i]=='}' && topElem!='{')
                return false;
        }
    }

    return StackEmpty(S);  //检索完全部括号后，栈空说明匹配成功
}
```

获取笔记